

Assignment: A Design Study

Telemetry Downlink for a CubeSat in Low Earth Orbit

Module 1, Source Coding Due 31 August

This is one problem worked from end to end, rather than a set of unrelated exercises. Everything you need has already been covered: the Kraft inequality, Huffman coding, the D-ary extension, Shannon–Fano–Elias, and the relative-entropy penalty for coding with the wrong model. What is new is the order in which you meet them, which is the order an engineer meets them rather than the order in which we taught them.

The satellite is invented; the numbers are not unreasonable. A scientific calculator is all you need. If you would rather write a few lines of code, that is fine, but hand the code in with your answers.

I would much rather see clear reasoning with an arithmetic slip in it than a correct number with no working. Please write as though you were explaining the design to a colleague who has not seen the problem.

The situation

A 3U CubeSat in a low Earth orbit produces a continuous stream of housekeeping telemetry, each record being one of eight event types. The records accumulate in an on-board buffer and are downlinked to a single ground station during the one visible pass the spacecraft gets each orbit. Whatever cannot be emptied during that pass is lost, because the next orbit's data is already arriving.

The link is modest and the pass is short.

Parameter	Value
Downlink	UHF, 9 600 bit/s
Usable pass duration	8 minutes (480 s)
Capacity of one pass	$9\,600 \times 480 = 4\,608\,000$ bits
Records buffered between passes	1 700 000
Operating modes	Sunlit (65% of the orbit), eclipse (35%)
Mode behaviour	Constant within a frame, and known on board

The event statistics differ between sunlight and eclipse, and that turns out to matter more than you might expect. In eclipse the spacecraft runs on battery and the thermal environment is changing, so thermal and power records dominate; in sunlight the housekeeping chatter is more routine.

Event type	p (sunlit)	p (eclipse)
E1 Housekeeping OK	0.36	0.18
E2 Attitude update	0.20	0.10
E3 Thermal reading	0.14	0.24
E4 Power and battery	0.11	0.26
E5 Payload status	0.08	0.06
E6 Link event	0.05	0.08
E7 Fault warning	0.04	0.05

Event type	p (sunlit)	p (eclipse)
E8 Critical fault	0.02	0.03

Where a single distribution is needed, use the mode-averaged one, $p_m = 0.65 p_s + 0.35 p_e$. Within a mode you may treat the source as memoryless.

What I would like you to do

Part A. Get a baseline

Build a binary Huffman code for the sunlit distribution. Show the merge sequence and draw the tree — by hand is perfectly acceptable, provided I can follow the order in which you combined things.

Report the codeword for each event, the length vector, the expected length L , the entropy H , and the efficiency. Check that your lengths satisfy the Kraft inequality, and say what the value you obtain tells you about the code.

Finish with one sentence: how much does this buy you per record, compared with simply numbering the eight event types in three bits?

Part B. Pay for the code table

Telemetry goes down in frames of N records. Each frame has to be decodable on its own — if one frame is lost, the next must still be readable — so every frame carries its own code table in a header.

The table is sent in canonical form. A canonical Huffman code is one whose codewords are assigned from the length vector by a rule both ends agreed on beforehand: shortest lengths first, and within a length, in symbol order. Because the rule is fixed, the receiver can rebuild every codeword from the lengths alone, so the header need only carry the eight lengths and not the codewords themselves. Each length goes into a fixed four-bit field.

Three assumptions, which I would like you to state in your answer rather than leave implicit:

- The header costs $8 \times 4 = 32$ bits, charged once per frame.
- The fixed-length code carries no header at all. Its mapping is settled at design time and never transmitted. This asymmetry is the whole point of the question.
- Take the frame to lie entirely in sunlight, so your Part A code applies with $L = 2.6100$ bits per record. Since L is an expected length, everything below is expected bits.

- (i) Write the expected bits per frame as a function of N , for the fixed-length code and for Huffman with its header.
- (ii) Find the smallest whole N at which Huffman becomes the cheaper of the two.
- (iii) Work out the net saving per record at $N = 50$, $N = 200$ and $N = 500$, and say what it tends to as N grows. Is an operational frame of 200 to 500 records a safe choice?
- (iv) Worth a minute of your time, though it is an aside: the header gives four bits to each length. What is the longest codeword a binary Huffman code on eight symbols could ever produce, and how wide does that field really need to be?

Part C. One code, or two?

Here is the design decision, and two proposals are on the table.

The first is to build a single Huffman code for the mode-averaged distribution p_m and use it all the time. The second is to build two codes, one per mode, and put a single mode flag at the head of each frame to say which applies. Both code tables would be agreed with the ground station in advance, so the second scheme sends no extra header — only that one flag bit.

Work out the expected length of each proposal. Report the saving of the second over the first, in bits per record and as a percentage, and find the shortest frame for which the flag bit pays for itself.

Part D. Put a number on using the wrong model

Suppose the single-code design is adopted and the spacecraft goes into eclipse. The code no longer matches the source. Compute the expected length it actually achieves on the eclipse distribution, and the penalty relative to two references: the eclipse-optimal Huffman code, and the eclipse entropy itself.

Now compute $D(p_e || p_m)$ and compare it with the second of those penalties. The two are close but not equal, and I would like you to say precisely why, which of them is larger, and what the difference consists of. This is the part that tells me whether Lecture 3 landed.

Part E. Consider a ternary link

A proposed radio upgrade would use a three-level line code, making the channel alphabet $D = 3$. Construct a ternary Huffman code for p_m .

Say how many dummy symbols you need, and justify the number from the divisibility condition rather than by trial. Report the length vector, L in ternary symbols per record, the ternary entropy H_3 , and the efficiency.

Then answer the question that actually matters: is the ternary link better than the binary one? Put both on a common basis before you decide, and be explicit about what you are assuming the channel is limited by. Your conclusion should depend on that assumption, and I expect you to say so.

Part F. Try Shannon–Fano–Elias

Build the SFE code for p_m . Tabulate $F(x)$, the modified CDF $\bar{F}(x)$, the length $l(x)$ and the codeword for each event. Report L , and check that it falls inside the band $H + 1 \leq L < H + 2$.

Then work out how many bits per pass this design would need, and say whether SFE is a serious candidate here.

Part G. Decide what to fly

For each of the four designs — fixed-length, single Huffman, two-code Huffman, and SFE — compute the total bits needed to downlink all 1 700 000 buffered records, and express that as a percentage of the pass capacity.

Then tell me which one you would fly, and defend it in no more than a page. I also want at least two things your analysis has ignored: effects that could change the answer, or risks the arithmetic hides. Say how you would go about testing for them. There is no single right answer here; what matters is the argument.

Two practical notes

A word about ties. At several points in this problem two nodes have equal probability and the merge order is genuinely ambiguous. Any consistent choice is fine. Different tie-breaks give different codeword length vectors, but they must give the same expected length — so if two of you disagree about L , at least one of you has made a mistake, and comparing your trees is the quickest way to find it.

Hand-drawn trees are welcome. Tables of codewords should be typed, or written neatly enough to read. For Part G, one page of prose is plenty; I am looking for judgement rather than volume.